

Side-channel safe conditional moves and swaps

David Santos^{1*} and Michael Scott²

¹ Instituto de Computação
Universidade Estadual de Campinas
`d261032@dac.unicamp.br`

² Cryptography Research Centre
Technology Innovation Institute
`michael.scott@tii.ae`

Abstract. Constant-time implementations are a cornerstone of secure cryptographic systems, particularly in the context of key exchange protocols and digital signature schemes. These implementations are designed to eliminate timing side-channel vulnerabilities by ensuring that the program’s execution time is independent of secret data. A fundamental building block for achieving constant-time behavior is the conditional move operation. Unlike traditional branching constructs (such as if statements), which may introduce data-dependent timing variations, conditional moves allow developers to write logic that behaves identically at the hardware level regardless of input values. As a result, they are widely used in cryptographic libraries and standards to ensure both functional correctness and resistance to timing attacks.

In this work, we describe our efforts to implement elliptic curve cryptography with some immunity against certain power leakage side-channel attacks, using standard C and Rust code.

Keywords: Side-Channel Analysis, Elliptic Curves, Countermeasures.

1 Introduction

Consider a library of functions in C or Rust that implements elliptic curve cryptography. Assume that extreme care has been taken to ensure that the code executes in constant time. That is our software implements straight-line code with no secret dependent branches or array indexing. Ideally the code might contain no “if” statements at all.

This is necessary, but not sufficient. Our compiler has its own agenda. So we must also ensure that the compiler, in its own ruthless pursuit of performance and in the interests of shaving a few clock cycles off the runtime, has not subverted our high level language intention by implementing our code with secret-dependent non-constant time assembly language. See [22], [4] and [21]. If necessary this may require examination of the generated assembly language, paying special attention

* This work was conducted while David Santos was doing an internship at Technology Innovation Institute, UAE.

to any branch instructions. Tedious, but doable. Finally we must check that the architecture will execute all the instructions that we generate in constant time, irrespective of the data they are processing. So while an ARM Cortex-M4 is OK, a Cortex-M3 is not (its integer multiplication instruction takes a number of clock cycles dependent on the numbers being multiplied).

However constant time implementation by itself does not guarantee side-channel immunity. Indeed a constant time implementation makes other types of side channel attack even easier, as an attacker will know precisely where in the programs execution to target vulnerable code and launch such an attack. So what further measures can we take in software to improve resistance against power or EM (Electro-Magnetic) side-channel leakage, which may reveal our secrets? This is important, as in some cases it has been demonstrated that a “single trace” attack may be possible, where the attacker needs only to access the leakage from a single activation of the code. Ideally we can mount a plausible defence without requiring a complete library re-write with significant performance penalties and negative impact on code complexity, portability and maintainability [3]. To make it interesting let us also consider a setting where randomness (as often used for masking or blinding countermeasures) is not available.

First we need to understand how such leakage can occur. The working assumption is that the power consumption of an instruction (or its EM radiation) depends on the number of bits that the instruction flips between 0 and 1. For example consider the simple act of writing to a memory location. Consider two cases, (a) where the number we are writing is unrelated to the number already there, and (b) where we are writing back the same number. In the first case on average half of the bits in the memory location will need to be flipped. In the latter case, none of them. We should therefore not be surprised that the first case scenario will consume more power. Since the root causes are architectural, we cannot do much to prevent this. An instruction may execute in constant time independent of the data it is processing. But clearly we cannot say the same about the amount of power it consumes.

More generally this kind of leakage can be explained by the Hamming distance model [8], where the power consumed is considered proportional to the Hamming weight of $A \oplus B$, where A is the current state and B the updated state of a register or memory location. This updating takes place in the context of every instruction executed. But a lot of bit-flipping also takes place in parts of the architecture that are hidden from the programmer, for example in pipeline registers [2]. Therefore power consumption traces are never flat and uniform, to the naked eye they appear random and noisy. The high level language programmer, who cannot even be sure of the architecture on which their code will eventually be run, cannot propose countermeasures based on detailed modeling of this noise. But embedded in that noise will be effects that are correlated to secrets. And here deep-learning methods are frighteningly capable of discerning patterns in the noise, that ultimately reveal secrets. In one recent work [7] it was discovered that by simply swapping the order of a pair of high-level language instructions a devastating side-channel leak could be either be

facilitated or blocked. Securing implementations in such a context is not going to be easy. We cannot completely guarantee success against a determined and well resourced attacker. The title of this relevant paper [3] includes the words “mission impossible?” for a reason!

Countermeasures available to us include masking or blinding of our code to heavily disguise our real intention. So we generate enough of our own noise that an attacker will fail, or any attacks become impossibly difficult to mount in a real-world setting.

But we can also try and minimize the impact of secret data on register contents and memory read/writes. And as it happens in elliptic curve cryptography we typically process secrets one bit at a time. So ideally it may be possible to program in such a way that each bit only ever has the impact of making a one bit difference in relevant Hamming weights. This would give us some hope that the impact of the bit on the generated noise will be minimal.

Any blinding value fulfils its task and then disappears from the calculation. Its purpose is to bury the contribution of a single secret bit. Here again we need to be wary of our compiler, which may remove our blinding if it is smart enough to determine that it does not actually affect the final outcome.

In elliptic curve cryptography power/EM side-channel weakness is most commonly found in the conditional move/swap functions, which move or swap elliptic curve points dependent on a secret bit. So the challenge is to come up with standard C or Rust functions which will behave as desired in all, or at least most, circumstances.

Conditional moves and the case of the RFC 7748 CSWAP function

Timing attacks are a form of side-channel attack in which an adversary infers secret information by precisely measuring the time taken to perform cryptographic operations. This powerful attack technique was first introduced by Paul Kocher in his seminal 1996 paper [14], where he demonstrated that even small variations in execution time could be exploited to recover secret keys. His results applied to widely used public-key algorithms, including RSA, Diffie-Hellman, and DSA.

As a countermeasure to timing-based vulnerabilities, it has been widely recommended to use conditional move operations instead of standard `if` statements when the condition depends on secret data. Today, virtually all modern key exchange protocols and digital signature schemes rely heavily on conditional moves to maintain constant-time execution. This practice extends to recent proposals for Post-Quantum Key Encapsulation Mechanisms (PQ-KEMs), many of which, despite their use of constant-time techniques, have been shown to be vulnerable to side-channel and fault attacks in several works [23,3].

In the context of Elliptic Curve Cryptography (ECC), the main timing vulnerability arises from the scalar multiplication operation, which is a fundamental building block in both key exchange protocols and digital signature schemes.

For example, in the *RFC 7748* memorandum [15], the Internet Research Task Force specified a Montgomery ladder-based scalar multiplication function, implemented over the elliptic curves `Curve25519` and `Curve448`, targeting the

128-bit and 224-bit security levels, respectively. The RFC outlines implementation details for this approach, including curve parameters and the ladder-based functions known as **X25519** and **X448**.

To ensure execution time is independent of the bits of the secret scalar, these functions rely heavily on the **cswap** function, which is defined in Algorithm 1. The function uses a constant-time conditional swap to avoid secret-dependent branches. The value **mask(swap)** is a word of all ones or all zeros (of the same bit length as the operands x_2 and x_3), and is typically computed as **mask(swap)** = $0 - \text{swap}$.

To compute elliptic curve point multiplication in a constant-time manner, the functions **X25519** and **X448** perform exactly 255 and 448 iterations, respectively, processing each bit of the scalar one at a time. At the end of every iteration, both functions invoke the **cswap** procedure as defined in Algorithm 1.

Algorithm 1 CSWAP(**swap**, x_2 , x_3)

```

1:  $dummy \leftarrow \text{mask}(\text{swap}) \text{ AND } (x_2 \text{ XOR } x_3)$ 
2:  $x_2 \leftarrow x_2 \text{ XOR } dummy$ 
3:  $x_3 \leftarrow x_3 \text{ XOR } dummy$ 
4: return  $(x_2, x_3)$ 

```

Unfortunately and as discussed above, despite its highly regular and constant-time design, the elliptic curve implementation specified in RFC 7748 remains inherently vulnerable to simple, low-bandwidth *single-trace* side-channel attacks.

In the following section, we review various solutions that have been proposed to strengthen conditional move operations, each with varying degrees of success. We then present our own approach to present a first line of defense against *single-trace* side-channel attacks.

Organization

The remaining of this paper is organized as follows, in Section 2 we go over some conditional move functions that are commonly used and are known to be vulnerable to power side-channel attacks. Then in Section 3 we propose our solution, which uses a constantly mutating mask to mitigate the difference of power consumption between executions and use tricks to prevent the compiler from removing our protections. In Sections 4 and 5, we present the results of our experiments. First, we use **ELMO**, an emulator-based power leakage simulator [18], to analyze potential side-channel vulnerabilities. We then execute the same code on a ChipWhisperer Lite development board to validate our findings on real hardware. Finally, in Section 6, we discuss the outcomes of these experiments, and in Section 7, we summarize our conclusions.

2 Basic solutions

There are two basic ideas recommended for a constant time move dependent on a single bit. The first is due to Bernstein [6]. If the bit is b then

$$X = (1 - b)X + bY$$

assigns Y to X if $b = 1$, and leaves X unchanged if $b = 0$. Another idea, as suggested in [15], would be to transform the bit b into a computer word that is either all 0s or all 1s. This can be as simple as assigning $D = -b$, where b is 0 or 1. Then

$$X = X \oplus ((X \oplus Y) \wedge D)$$

The two ideas are basically equivalent as that last can be converted to

$$X = (X \wedge \neg D) \oplus (Y \wedge D)$$

We will refer to the former method as an arithmetic conditional move, and the latter as a logical conditional move. While both of these methods allow a constant time move, they do not address the power leakage issue.

In practice the logical conditional move is by far the more widely used, as for example in the Go language crypto/subtle package, or the Rust equivalent as implemented in the curve25519-dalek crate [17] which is used by the popular Signal protocol. The logical conditional move might also be considered to be inherently faster, as it does not require a potentially expensive multiply operation.

A single trace EM side-channel attack on the logical conditional move is described by Alam et al. [1]. They propose a countermeasure using a random blinding value R .

$$M = (X \oplus Y) \wedge D$$

$$M = M \oplus R$$

$$X = (X \oplus M) \oplus R$$

They also demonstrate an awareness of potential compiler misbehaviour, and suggest a method to ensure that the compiler generates code which correctly implements the intention. Nevertheless the first instruction still has a problem ANDing with an all-zeros or all-ones D . This issue was recognised by Genet and Kaludervic [10] section 6.1, who suggested splitting D into a pair of randomly chosen parts such that $D = D_1 \oplus D_2$, which are then applied independently.

In their paper [16] Loiseau et al. suggest that Bernstein’s arithmetic conditional move could be replaced by

$$X = ((1 - b) + r_0)X + (b + r_1)Y - r_0X - r_1Y$$

Now all multiplications are by random values derived from two independent random blinding values r_0 and r_1 . This idea is described by [3] as an “effective

countermeasure”. Note that the calculation may cause numerical overflows, but it will still work as intended.

But there is another potential problem. The whole point of a conditional move is to either replace the original value with a new one, or leave it unchanged. Leaving it unchanged involves writing back the original value to its memory location. The power consumption required for this write-back is dependent on the number of bits that have changed. And if its the same value then no bits will have changed. And this may be detected [3]. Note that this weakness completely by-passes any blinding we might have attempted in the calculation of the write-back value.

A clever protocol-level solution is suggested in [3]. An elliptic curve point in standard projective coordinates, for example represented as (X, Y, Z) , can be randomized as (rX, rY, rZ) using a random value r , without changing its real affine value $(X/Z, Y/Z)$. So the trick is to always randomize in this way before writing back to memory.

3 Our Solution

We prefer a less costly solution based on the arithmetic approach as (a) multiplication probably generates more useful “noise” and (b) the logical method “amplifies” the single bit b into a full word, which we suspect may make its impact more vulnerable to detection [19]. Better to whisper a secret rather than shout it. See section III-A of [20] and section 6.1 of [11].

Our proposed solution is based on that of Loiseau et al. First we simplify it by just using one blinding value r rather than two. Next, as we are not convinced of the need for true randomness, but nevertheless would like r to change, we deploy a very simple additive generator that on a 32-bit processor increments r by $5AA5A55A_{16}$ on every function call. This value is not critical, and was chosen simply as a random even 32-bit number with an average Hamming weight. As made explicit by the Rust implementation this process is not thread-safe. Another thread may jump in and increment r . But this is not an issue as the value of r is ultimately removed from the calculation.

$$X = ((1 - b) + r)X + (b + r)Y - r(X + Y)$$

There are a couple of issues with this. First an optimizing compiler can strip out r , reverting the generated code back to the original simpler Bernstein form. Indeed we have confirmed that this is exactly what the clang optimizer does. Secondly we need to deal with the consequences of writing the same value back to memory.

Addressing the last problem first, we will (ab)use the “volatile” keyword supported by the C and Rust compilers. In an embedded environment (which as luck would have it is the domain we are mostly concerned with), memory writes can take place to memory-mapped output registers which actually control physical devices. In this setting all such writes must obviously be honoured and not

optimized away by the compiler. To exploit this we will break our Loiseau et al. based solution into two stages.

$$T = X$$

$$X = ((1 - b) + r)T + (b + r)Y$$

$$X = X - r(T + Y)$$

By marking X as volatile, the compiler will now always perform both writes to X , and we are never writing back the same value to its original location. This also solves our first problem. Since the first write must now take place, the compiler will no longer optimize away the blinding, as it will not benefit from doing so. The exploitation of volatility to block unwanted compiler optimizations was suggested by Bernstein [5].

It is the intention of the code as written that the bit b and its inverse are added directly to r . Note that the last bit of r is 0, so this addition will not cause any carry propagation. Random noise is contributed to the calculation not so much from r , but more from the data being processed, from X and Y .

Instead of coding $(1 - b) + r$ directly, we can obfuscate the calculation using our awareness (not available to the compiler) that b takes on only two values, 0 or 1. So we calculate it as $(\neg b) \wedge (r + 1)$. This will further frustrate any attempt by the compiler to simplify the code. Calculating in this way also makes it clearer that the impact of b on the Hamming weight of operands is only that of a single bit. In our proposed solution we are interested in conditionally moving an array of length N which represents a multiprecision number into another array. It is common practice with modern fast implementations of elliptic curve cryptography that N , a function of the elliptic curve and the processor wordlength, is fixed at compile time. This facilitates aggressive loop unrolling by the compiler particularly when compiler optimization is set to maximum.

In our case the multiprecision number represents a coordinate of a point on an elliptic curve. In Figure 1, we present the C and Rust implementations of our proposed `cmove` routine, which integrates all the aforementioned techniques.

Despite these measures we cannot be entirely sure that the compiler has not subverted our efforts. It will be advisable to take language-specific/compiler measures to ensure that the function will not be inlined, in case the compiler might be able to detect that b and its inverse are only single bits. If so it may be tempted to substitute a branch. Some experiments with the latest versions of clang and rustc show that, assuming the bit is not known at compile time, the attributes `noinline` in C/C++ and `#[inline(never)]` in Rust are honoured by the compilers. However this is not absolutely guaranteed, and should always be checked. One extreme measure to future-proof against compiler developments would be put the function in its own separately compiled dynamically-linked module, and hence not visible to the calling program when it is being compiled. In general the less the compiler knows about the context in which the function is to be used, the better. A very useful resource is the site www.godbolt.org which allows the generated assembly language to be examined for a variety of

listing optionsC code

```
// 32-bit C/C++ code for conditional move of g to f if b=1

#include <stdint.h>

void cmove(int b, const uint32_t *g, volatile uint32_t *f) {
    int i;
    uint32_t c0,c1,s,t,r,aux;
    static uint32_t R=0;
    R+=0x5aa5a55a;
    r=R;
    c0=(~b)&(r+1); // (1-b)+w;
    c1=b+r;
    for (i=0;i<N;i++) {
        t=f[i]; s=g[i];
        f[i]=aux=c0*t+c1*s;
        f[i]=aux-r*(t+s);
    }
}
```

listing optionsRust code

```
// Embedded Rust (in release mode)
#[inline(never)]
pub fn cmove(b: u32,g: &[u32],f: &mut [u32]) {
    static mut R:u32=0;
    let r:u32;
    unsafe {
        R+=0x5aa5a55a;
        r=R;
    }
    let c0=(!b)&(r+1);
    let c1=b+r;
    for i in 0..N {
        let s = g[i]; let t = f[i];
        unsafe{core::ptr::write_volatile(&mut f[i],c0*t+c1*s)}
        f[i]-=r*(t+s);
    }
}
```

Fig. 1: C and Rust code for the proposed cmove solution.

architectures and compilers, using various optimization options. If all has gone according to plan the function will not be inlined, and the generated code for the loop should include three integer multiplication instructions, and two memory writes. This can easily be confirmed.

For example Figure 2 shows the assembly language output of the gcc 15.2 compiler for the loop with -O2 optimization for the Cortex M4. The use of the `aux` variable in the C code suppresses a redundant memory reload following the first memory write, which can be an unwanted side effect of the volatility property, as the stored value is read back again just in case an I/O device may have changed it [9].

It is an interesting exercise to remove the volatile keyword from the C code, re-compile, and observe the difference. Observe that with the volatile keyword removed there is just one memory write instruction.

```
.L2:
    ldr    r3, [ip, #4]!
    ldr    r0, [r2]
    cmp    ip, r5
    add    r1, r0, r3
    mul    r3, r6, r3
    mla    r3, r0, r4, r3
    mls    r1, lr, r1, r3
    str    r3, [r2]
    str    r1, [r2], #4
    bne    .L2
```

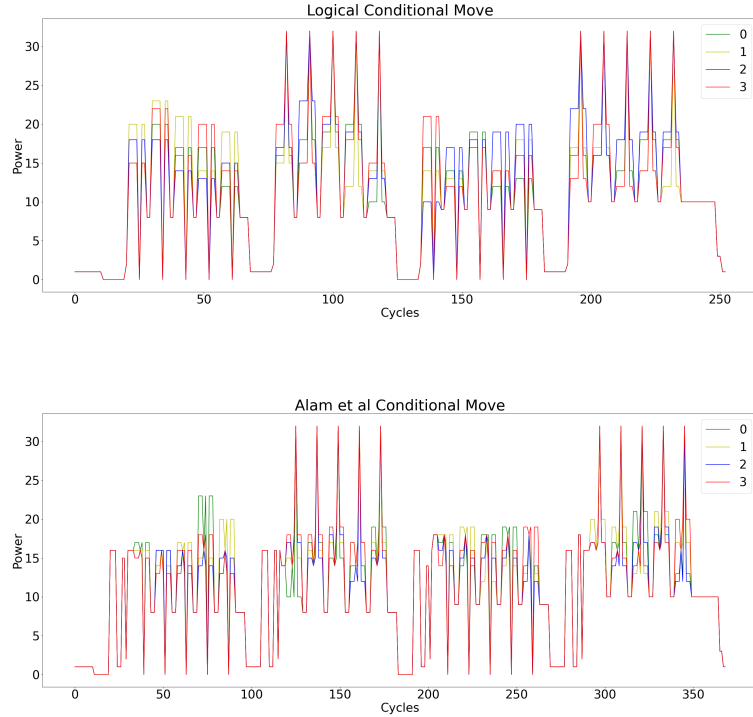
Fig. 2: Cortex M4 assembly code for `cmove` loop.

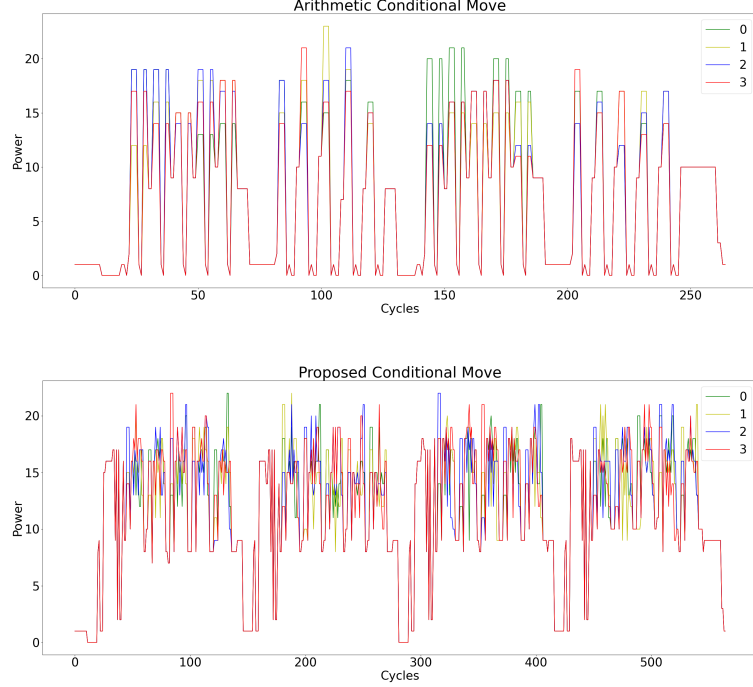
```
.L2:
    ldr    r1, [ip, #4]!
    ldr    r3, [lr, #4]!
    cmp    r5, ip
    add    r4, r1, r3
    mul    r3, r6, r3
    mls    r3, r2, r4, r3
    mla    r3, r1, r0, r3
    str    r3, [ip]
    bne    .L2
```

Fig. 3: Cortex M4 assembly code for `cmove` loop – without *volatile*.

4 Test results

Recall that we are primarily attempting to provide a constructive defense against (at a minimum) the single trace attacker. To get some validation for the effectiveness of our countermeasure we use the ELMO power leakage simulator [18] for the 32-bit Cortex M0 processor, with its 3-stage pipeline, using the Hamming distance model for power leakage. The C code with $N = 5$ is compiled using gcc -O2 optimization. We choose $N = 5$ simply to make our graphs more accessible. We measure the simulated power consumption for a conditional move with four random inputs, for the four sequential values of $b = 0, 1, 0, 1$. A single trace attacker gets only one shot at it, so the hope will be that a clear cut distinction will not be visible between the two cases of 0 and 1. We consider four types of conditional move using (a) the simple logical method (RFC7748), (b) the same with the Alam et al. [1] masking countermeasure, (c) the simple arithmetic method, and (d) our proposed method.





As can be seen the Alam et al. countermeasure is completely ineffective. A close examination of the assembly language output shows that the gcc compiler has copied the register associated with the random mask to another working register for use inside of the loop. It then takes measures which ensure that the original register is protected and not “optimized out”. Unfortunately no protection is afforded to the working register, which is cleared to zero in a bid to optimize the code. Hence the masking value has no effect.

5 Tests on Hardware

In addition to the experiments conducted using ELMO, we also carried out side-channel analysis on a real device using a ChipWhisperer Lite development board. ChipWhisperer platforms are widely used for side-channel research and provide a practical means of evaluating cryptographic implementations under realistic attack conditions. The Lite version, in particular, is relatively low-cost, offering insights into the capabilities of a resource-constrained adversary. Our setup featured a ChipWhisperer Lite with an embedded STM32F303 microcontroller, based on an ARM Cortex-M4 core, which served as the target device for our experiments.

5.1 Tests setup

To compile the source code and generate the corresponding hex files for execution on the target device, we used arm-none-eabi version 14.2.Rel1, running on a MacBook Pro. The source code had to be adapted to conform to the ChipWhisperer project format to enable communication via the SimpleSerial protocol. This protocol was used to send commands signaling the start of measurements and to transmit the input data for the conditional move operations, among other functionalities.

As a reference, we used the `simpleserial-base.c` example provided in the ChipWhisperer GitHub repository³, and integrated our custom `cmove` functions into it. For building the code, we relied on the provided makefiles, which already include the appropriate compiler flags. We modified the `Makefile.inc` by removing the default debug flags and changing the optimization level from `-Os` (optimize for size) to `-O2` (optimize for speed), to better match our performance analysis requirements.

We used a Python script to configure the devices, load the hex files, and capture the power traces. For simplicity and consistency, we opted to use the default configuration provided by the ChipWhisperer Python library. In this setup, the target device operates at a frequency of 29.54 MHz, while the ChipWhisperer clock is set to four times the target frequency. Each captured trace consists of 5000 samples, corresponding to approximately 1250 clock cycles of the target device. The Python scripts used for both trace collection and subsequent analysis will be made publicly available in a GitHub repository upon acceptance of this work.

Figure 4 presents a full power trace of the Logical Conditional Move operation, executed with arbitrary input values and a secret bit set to 1. The trace was captured using our measurement setup. The segment following the red dotted line corresponds to the device idling after the completion of the `cmove` function, illustrating that only a small portion of the captured trace reflects the actual function execution. By examining the corresponding assembly code, we can estimate the number of clock cycles required to execute the conditional move routine. Although this number varies depending on the specific implementation, it typically does not exceed a few dozen clock cycles, which is significantly less than the total number of samples captured per trace.

We also used the functionality provided by the Data Watchpoint and Trace unit in the STM32F303 target device to measure the number of clock cycles necessary to perform each of the conditional move functions. The values measured are similar to the estimated values and are shown in Table 1. It is also worth noting that considering only the instructions executed in the conditional move function, some of them are independent of the secret data and do not contribute to the attack.

³ <https://github.com/newaetech/chipwhisperer>

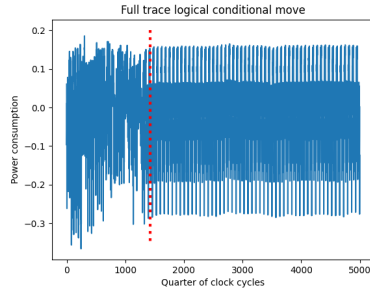


Fig. 4: Full trace with one execution of the logical conditional move.



Fig. 5: Full trace with four executions of the logical conditional move alternating the secret bit between values 0 and 1.

Method	Logical	Arithmetic	Proposed
Number of clock cycles	69	66	100
Number of samples per trace	276	264	400

Table 1: Clock cycles necessary to perform each of the conditional move functions.

5.2 Results

Since the optimization flag -O2 was found to remove the protective mechanism proposed by Alam et al., we excluded it from this part of the analysis. For the remaining methods, our initial approach mirrored that used in the ELMO-based experiments: we captured four traces, each containing four consecutive executions of the conditional move function using random inputs, with the secret bit alternating as $b = 0, 1, 0, 1$. The resulting power trace for the Logical Conditional Move is shown in Figure 5. To include four function executions within a single trace, we had to relocate the trigger signals in the source code. This introduced additional operations resulting in significantly longer traces. Compared to the simulated ELMO environment, it became considerably more difficult to visually distinguish the influence of the secret bit across executions—even for the implementation that is expected to be the most vulnerable. As a result, we were compelled to revise our attack model for this setting and change our methodology as follows. We consider two scenarios:

1. The attacker analyzes all the instructions in a conditional move execution
2. The attacker is able to distinguish points of interest from points unrelated to secret data

In both experimental scenarios, we assume that the attacker has limited access to the target device, allowing them to collect only 256 traces of the cmove function. For each trace, the attacker does not know the input values or the corresponding secret bit. In the second scenario, we further assume that the attacker has access to an oracle that reveals the Points of Interest (POIs) for each method. In our analysis, we selected these POIs using a white-box approach.

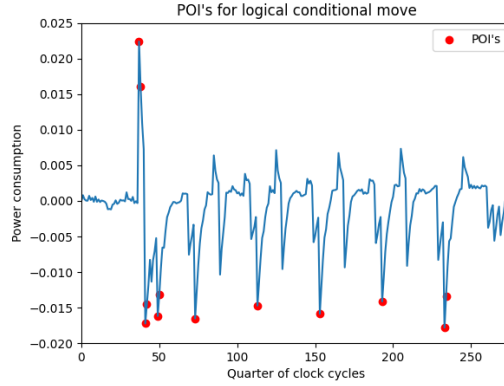
All input data used in the experiments were randomly generated using the `random.bytes` function from the NumPy Python library. To analyze the traces, we applied Principal Component Analysis (PCA), as implemented in `scikit-learn`, to reduce the dimensionality of the power traces—from thousands of samples per trace down to two principal components. We then visualized the resulting data by plotting the points corresponding to each trace execution: red dots represent executions with secret bit $b = 0$, and green dots represent those with $b = 1$. Our goal is to assess whether these two classes of points are spatially distinguishable, which would indicate a potential leakage of secret-dependent information.

The procedure followed for the first scenario is outlined below:

1. Generate 128 random input triples of the form $(0, X, Y)$ and 128 of the form $(1, X, Y)$, where X and Y are random inputs and the first element indicates the value of the secret bit b .
2. For each conditional move implementation:
 - (a) Capture one power trace corresponding to each input triple.
 - (b) Trim each trace to isolate the execution of a single cmove operation.
 - (c) Apply Principal Component Analysis (PCA) to reduce the dimensionality of the trace data.
 - (d) Visualize the resulting data by plotting the two principal components.

The results of this analysis are presented in Figure 8.

In the second scenario, the attacker is assumed to have access to a predefined set of Points of Interest (POIs). The procedure is identical to that described in the first scenario, except for step 3: instead of extracting the entire function execution from each trace, only the POIs are retained for analysis. To determine the POIs, we used an additional set of 256 traces, divided into two groups: G_0 , containing traces where the secret bit $b = 0$, and G_1 , where $b = 1$. This labeling was feasible because the analysis was conducted under a white-box model. For each group, we computed the mean trace across all samples, resulting in the vectors $\text{mean}(G_0)$ and $\text{mean}(G_1)$. We then computed the difference vector $\text{dif} = \text{mean}(G_1) - \text{mean}(G_0)$. POIs were selected by identifying the sample indices i for which $|\text{dif}_i|$ is large, as these samples are likely to correspond to instructions with high information leakage. The values of dif and the corresponding selected POIs for each function are illustrated in Figure 6.



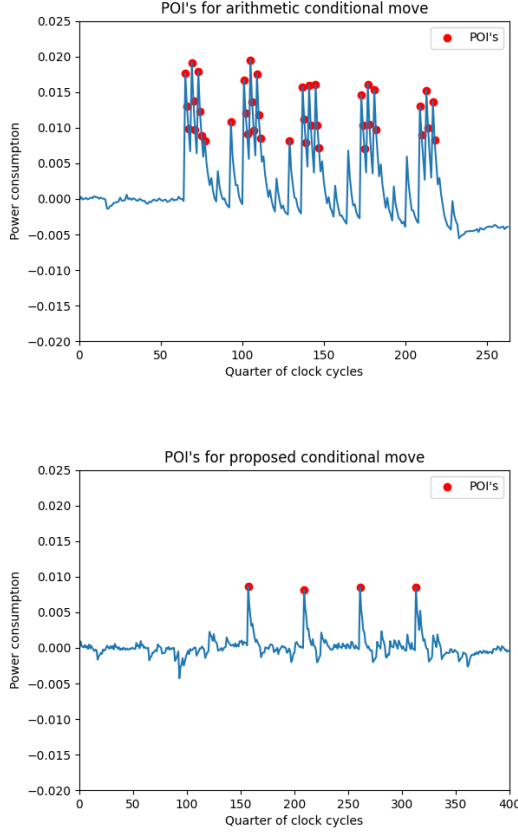


Fig. 6: Difference of means between groups of traces with secret bit equal to 0 and with secret bit equal to 1 for the conditional moves tested. The value of the secret bit is assumed to be known. The red dots correspond to the points of interest, where we assumed the leakage was most significant.

The results of the PCA for all the methods studied is shown in Figure 7.

Finally, it is important to highlight that our experimental setup differs significantly from real-world attack conditions. In our case, we employ hardware triggers to precisely mark the beginning of each conditional move function execution and capture power measurements with a resolution of quarter clock cycles. In contrast, a realistic adversary, such as in the attack described by Alam et al. [1], would not have access to such fine-grained timing information.

Moreover, our analysis is based on only 256 traces, each corresponding to a single execution of a conditional move. This is consistent with the amount of information an attacker would obtain from observing a single scalar-by-point multiplication using a 256-bit elliptic curve key. Remarkably, even with this limited data and a basic PCA-based analysis, our attacker is able to clearly distinguish

between traces with secret bit $b = 0$ and $b = 1$ in the logical and arithmetic conditional move implementations. However, this is not the case for our proposed method, which remains indistinguishable under the same conditions, as shown in Figure 8.

Increasing the attacker capabilities, allowing them to capture 256 traces of conditional move executions where he knows the value of the secret bit makes it possible for them to better distinguish the secret information used in different executions in our proposed method. However, acquiring such information is not trivial. First, in order to be able to do so, it would be necessary to have complete access to an exactly identical device in a pre-attack phase, this already increases the financial cost and the amount of time and effort an attacker has to spend. Second, even buying a device with the same model and same specifications as the target one might not be enough. Aspects such as the amount of time the target device has been used and environmental conditions such as temperature, humidity could create variations in the power consumption making the traces captured prior to the attack not so reliable. With that said, we believe that our proposed method is an improvement in the security against side-channel attacks for conditional move/swap functions, at a cost of roughly 70% increase in execution time for each conditional move/swap execution, which becomes less relevant when we compare with the cost of a full elliptic curve point multiplication. In one experiment we measured an overall 5% percent increase in the runtime of an implementation of an ECDH protocol when our proposed conditional swap was substituted for the simple logical conditional swap recommended there [15]. And this is in the context of a Montgomery ladder, where conditional swaps are used quite extensively

6 Discussion

Side-channel attacks have been a persistent concern for cryptographic developers since the seminal works on timing attacks[14] and power analysis[13]. Today, writing constant-time code is considered an essential skill for experienced cryptographers. Despite this awareness, and the existence of highly secure implementations such as [3], side-channel vulnerabilities remain a relevant and challenging issue. Why does this problem persist?

One major reason is cost: implementing robust protections against side-channel attacks often requires significant computational or engineering overhead, making some developers more willing to “take the risk” rather than incur the performance or complexity penalty. Another key issue is the difficulty of ensuring that the code written by the developer is precisely what gets executed on the hardware. In particular, compilers pose a critical obstacle: they can optimize away carefully crafted side-channel countermeasures, especially when those rely on subtle instruction-level behavior.

Indeed, the battle with compilers is often seen as an uphill one. Many mitigation strategies turn out to be palliative—working temporarily until a newer compiler version inadvertently breaks them. For example, the current version of

arm-none-eabi-gcc (as of this writing) already removes the protections proposed in [1], and it may only be a matter of time before newer compilers begin to undermine even more advanced countermeasures.

Our solution compared to both arithmetical and logical unprotected implementations look like an improvement. Indeed, in the code the secret bit b and its inverse manifest explicitly only in their addition to the least significant bit of r . If this would be detectable from some side-channel attack, then we might safely conclude that this indeed is “mission impossible”. In addition, up to this point, the tricks used to prevent the compiler from optimizing away the protection against side-channel attacks are effective.

Our experimental results suggest that if an attacker is able to identify points of interest (POIs) in the traces, this significantly improves their ability to distinguish between secret bits with values 0 and 1. It is also plausible that more sophisticated statistical techniques or simply capturing a larger number of traces could further enhance the attacker’s capacity to recover secret information.

Additionally, a limitation of our current approach is that the blinding values are derived deterministically from the processed data. As a result, repeated executions with the same inputs will yield the same internal values, potentially leaking information across traces. This deterministic behavior can be mitigated by applying a one-time projective-coordinate re-randomization to all input points. In fact, performing such re-randomizations more frequently—ideally before every scalar multiplication—is a well-established and powerful complementary countermeasure, as discussed in [3].

7 Conclusion

There is a new onus on cryptographic software developers: Not only to ensure that critical code paths execute in constant time, but also to deploy low-cost countermeasures capable of resisting powerful attackers, including those who can succeed with single-trace side-channel observations. As highlighted in [12], “we argue (for) the need for single-trace side-channel aware software development”—a view that emphasizes proactive design practices rather than reactive patching.

A key point in mitigating such attacks is identifying code structures that introduce data-dependent variations in power consumption, especially those that generate significant swings in Hamming weight based on secret values. These variations are typically exploited by attackers to extract sensitive information using only a small number of measurements, often under minimal assumptions.

In this work, we focused on one such critical operation: the conditional move. Widely used in modern cryptographic software to eliminate branches and enforce constant-time behavior, conditional moves are intended to avoid timing-based leakage. However, as we demonstrated, their implementation at the machine level still leaves them vulnerable to power analysis unless carefully crafted.

We proposed a new side-channel hardened conditional move routine that maintains functional correctness while minimizing leakage. Our solution is designed to behave identically at the hardware level for all possible (secret) input

values, resisting even single-trace side-channel attacks. We evaluated our design under both simulated and real-world conditions, showing that while traditional implementations exhibit distinguishable patterns that leak secret information, our proposed version provides a significantly higher level of resistance.

Acknowledgments. We would like to express our sincere gratitude to Francisco Rodríguez-Henríquez for his valuable contributions to this paper. His active participation including suggestions and discussions regarding the content and the structure of the paper were essential for the final state of this work.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alam, M., Yilmaz, B.B., Werner, F., Samwel, N., Zajic, A.G., Genkin, D., Yarom, Y., Prvulovic, M.: Nonce@once: A single-trace EM side channel attack on several constant-time elliptic curve implementations in mobile platforms. In: IEEE European Symposium on Security and Privacy, EuroS&P 2021, Vienna, Austria, September 6-10, 2021. pp. 507–522. IEEE (2021)
2. Barengi, A., Breveglieri, L., Pelosi, G.: Exploring Cortex-M microarchitectural side channel information leakage. *IEEE Access* **9** (2021)
3. Batina, L., Chmielewski, L., Haase, B., Samwel, N., Schwabe, P.: SoK: SCA-secure ECC in software - mission impossible? *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2023**(1), 557–589 (2023)
4. Bernstein, D.J.: Clang vs. Clang (2024), <https://blog.cr.yp.to/20240803-clang.html>
5. Bernstein, D.J.: Compiler-introduced timing leak in kyber reference implementation (2024), <https://tinyurl.com/pqc-forum-compiler>
6. Bernstein, D.J.: Curve25519: New diffie-hellman speed records. In: Yung, M., Dodis, Y., Kiayias, A., Malkin, T. (eds.) *Public Key Cryptography - PKC 2006*, 9th International Conference on Theory and Practice of Public-Key Cryptography. *Lecture Notes in Computer Science*, vol. 3958, pp. 207–228. Springer (2006)
7. Berzati, A., Viera, A.C., Chartouny, M., Vigilant, D.: Simple power analysis assisted chosen cipher-text attack on ML-KEM. *IACR Cryptol. ePrint Arch.* p. 2051 (2024), <https://eprint.iacr.org/2024/2051>
8. Brier, E., Clavier, C., Olivier, F.: Correlation power analysis with a leakage model. In: Joye, M., Quisquater, J. (eds.) *Cryptographic Hardware and Embedded Systems - CHES 2004: 6th International Workshop*. *Lecture Notes in Computer Science*, vol. 3156, pp. 16–29. Springer (2004)
9. Filho, D.L.G.: Personal communication (2025)
10. Genêt, A., Kaluderovic, N.: Single-trace clustering power analysis of the point-swapping procedure in the three point ladder of Cortex-M4 SIKE. In: Balasch, J., O’Flynn, C. (eds.) *Constructive Side-Channel Analysis and Secure Design - 13th International Workshop, COSADE 2022*. *Lecture Notes in Computer Science*, vol. 13211, pp. 164–192. Springer (2022)
11. Guerreau, M., Rossi, M.: A not so discrete sampler: Power analysis attacks on HAWK signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2024**(4), 156–178 (2024)
12. Karabulut, E., Alkim, E., Aysu, A.: Single-trace side-channel attacks on ω -small polynomial sampling: With applications to NTRU, NTRU prime, and CRYSTALS-DILITHIUM. *Cryptology ePrint Archive*, Paper 2022/494 (2022), <https://eprint.iacr.org/2022/494>
13. Kocher, P., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener, M. (ed.) *Advances in Cryptology — CRYPTO’99*. pp. 388–397. Springer Berlin Heidelberg, Berlin, Heidelberg (1999)
14. Kocher, P.C.: Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In: Koblitz, N. (ed.) *Advances in Cryptology — CRYPTO ’96*. pp. 104–113. Springer Berlin Heidelberg, Berlin, Heidelberg (1996)
15. Langley, A., Hamburg, M., Turner, S.: Elliptic Curves for Security. RFC 7748 (2016)
16. Loiseau, A., Lecomte, M., Fournier, J.: Template attacks against ECC: practical implementation against Curve25519. In: *HOST 2020 - IEEE International Symposium on Hardware Oriented Security and Trust* (2020)

17. Lovecruft, I., de Valence, H.: A pure-rust implementation of group operations on ristretto and curve25519 (2024), <https://github.com/dalek-cryptography/subtle/blob/main/src/lib.rs>
18. McCann, D., Oswald, E., Whitnall, C.: Towards practical tools for side channel aware software engineering: grey box modelling for instruction leakages. In: 26th USENIX security symposium (USENIX Security 17). pp. 199–216 (2017)
19. Nascimento, E., Chmielewski, L., Oswald, D.F., Schwabe, P.: Attacking embedded ECC implementations through cmov side channels. IACR Cryptol. ePrint Arch. p. 923 (2016)
20. Qiu, J., Aysu, A.: SHIFT SNARE: Uncovering secret keys in FALCON via single-trace analysis. Cryptology ePrint Archive, Paper 2025/146 (2025), <https://eprint.iacr.org/2025/146>
21. Schneider, M., Lain, D., Puddu, I., Dutly, N., Capkun, S.: Breaking bad: How compilers break constant-time implementations. arXiv:2410.13489 (2024)
22. Simon, L., Chisnall, D., Anderson, R.J.: What you get is what you C: controlling side effects in mainstream C compilers. In: 2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018. pp. 1–15. IEEE (2018)
23. Ueno, R., Xagawa, K., Tanaka, Y., Ito, A., Takahashi, J., Homma, N.: Curse of re-encryption: A generic power/em analysis on post-quantum kems. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2022**(1), 296–322 (2022)

Conditional swap code

```
#include <stdint.h>
void cswap(int b,volatile uint32_t *g,volatile uint32_t *f) {
    int i;
    uint32_t c0,c1,s,t,r,v,aux;
    static uint32_t R=0;
    R+=0x5aa5a55a;
    r=R;
    c0=(~b)&(r+1);
    c1=b+r;
    for (i=0;i<N;i++) {
        t=f[i]; s=g[i];
        v=r*(t+s);
        f[i]=aux=c0*t+c1*s;
        f[i]=aux-v;
        g[i]=aux=c0*s+c1*t;
        g[i]=aux-v;
    }
}

// Embedded Rust
#[inline(never)]
pub fn cswap(b: u32,g: &mut [u32],f: &mut [u32]) {
    static mut R:u32=0;
    let r:u32;
    unsafe {
        R+=0x5aa5a55a;
        r=R;
    }
    let c0=(!b)&(r+1);
    let c1=b+r;
    for i in 0..N {
        let s = g[i];
        let t = f[i];
        let v=r*(t+s);
        unsafe{core::ptr::write_volatile(&mut f[i],c0*t+c1*s)}
        f[i]-=v;
        unsafe{core::ptr::write_volatile(&mut g[i],c0*s+c1*t)}
        g[i]-=v;
    }
}
```

A Experiments with Principal Component Analysis using 256 traces

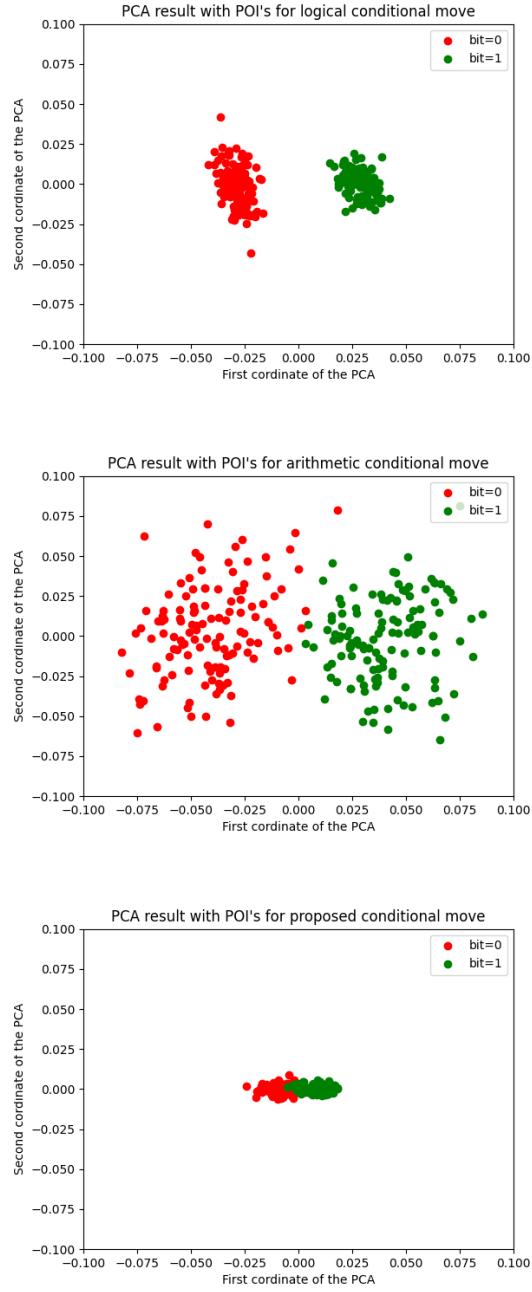


Fig. 7: PCA applied to 256 traces, each containing only the points of interest of an execution of a conditional move function. The red dots correspond to traces with secret bit equal to 0 and the green dots correspond to secret bit equal to 1.

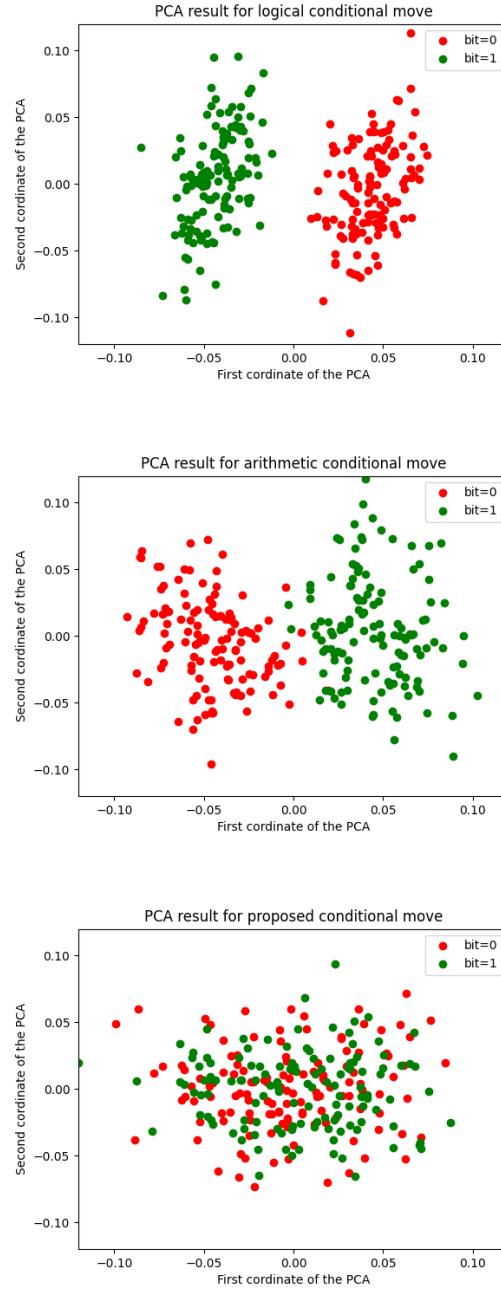


Fig. 8: PCA applied to 256 traces, each containing one execution of a conditional move method. The red dots correspond to traces with secret bit equal to 0 and the green dots correspond to secret bit equal to 1.